



COMPILER DESIGN • CSA1401

Lexical Analysis and the Role of the Lexical Analyzer

Presented by:

Goutham Enamala - 192311190

Introduction to Compiler

A compiler is a program that bridges human-readable code and machine execution.



Translates

Converts high-level language source code into machine code the CPU can execute.



Detects Errors

Identifies lexical, syntax, and semantic errors before the program ever runs.

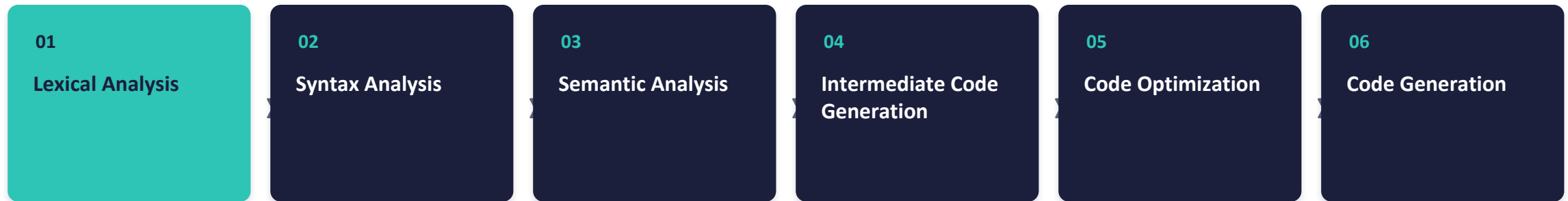


Optimizes Code

Improves performance and efficiency of the generated machine code.

Compiler Phases

Six sequential phases transform source code into executable machine code.



Focus of this presentation: Lexical Analysis is the first phase — every later phase depends on the tokens it produces.

What is Lexical Analysis?

-  First phase of the compiler
-  Converts characters into tokens
-  Removes whitespace and comments
-  Sends the resulting tokens to the parser

SOURCE CODE

```
int a = 10;
```

↓ *lexical analyzer*

int	a	=	10	;
-----	---	---	----	---

TOKENS

Ready for syntax analysis



Role of the Lexical Analyzer

Four core responsibilities carried out during scanning.



Reads Source Code

Scans the program character by character from the input file.



Generates Tokens

Groups characters into meaningful lexemes and classifies them as tokens.



Detects Lexical Errors

Flags invalid identifiers, illegal symbols, and malformed constants.



Maintains Symbol Table

Records identifiers along with their type, scope, and memory details.



Engineer to Excel



Tokens

A token is a categorized unit of meaning produced by the lexical analyzer.

Example:

```
int a = 10;
```



Keyword

`int`



Identifier

`a`



Operator

`=`



Constant

`10`



Separator

`;`



Engineer to Excel



Token Types

Every token produced by the lexical analyzer falls into one of five categories.



Keywords

Reserved words with fixed meaning, e.g. int, if, return.



Identifiers

Names given to variables, functions, and arrays.



Operators

Symbols denoting operations, e.g. +, =, &&.



Constants

Fixed literal values such as numbers and characters.

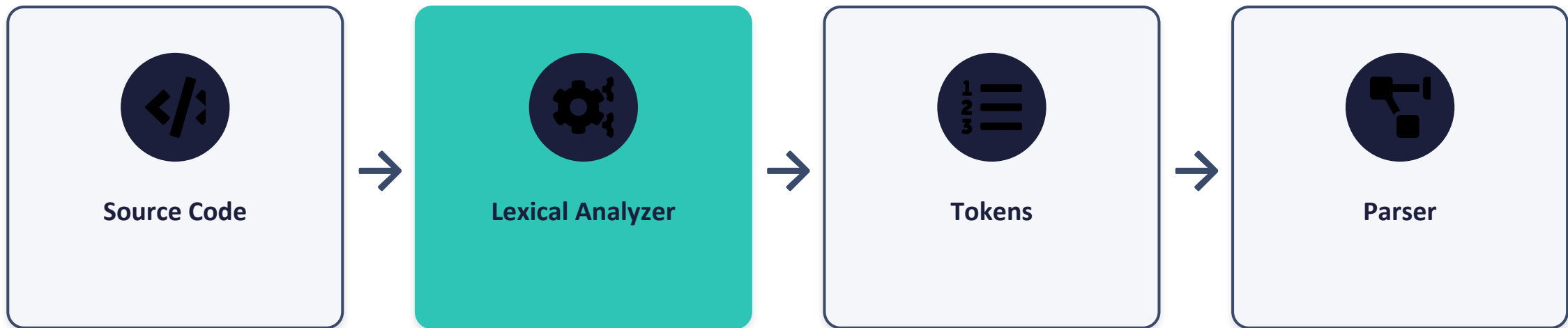


Separators

Punctuation marks like ; , () { }.

How Lexical Analysis Works

Source code flows through the analyzer and emerges as a token stream for the parser.





Engineer to Excel

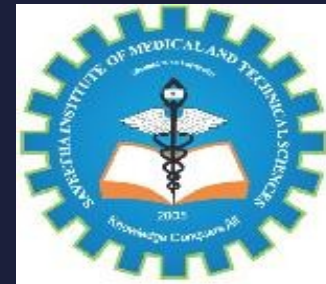
Symbol Table

A data structure built during lexical analysis to track every identifier in the program.



Identifier	Type	Scope	Memory Location
a	int	local	0x7ffde42

Why it matters: Later compiler phases query the symbol table to check types, resolve scope, and allocate memory correctly.



Lexical Errors

Common mistakes the lexical analyzer catches before parsing even begins.



Invalid Identifier

A name that breaks naming rules, such as starting with a digit.

```
2var = 5;
```



Illegal Symbol

A character that does not belong to the language's alphabet.

```
a @ b;
```



Unclosed String

A string literal missing its terminating quotation mark.

```
"hello
```

Advantages



Faster Compilation

Simple, specialized scanning is far quicker than parsing raw characters directly.



Early Error Detection

Lexical mistakes surface immediately, before wasting time on later phases.



Simplifies Parsing

The parser works with clean tokens instead of raw, unstructured text.

Applications



Compilers

C, C++, Java, and Python compilers all begin with a lexical scanning stage.



IDEs

Editors use lexers for syntax highlighting and real-time error underlining.



Interpreters

Scripting language interpreters tokenize each line before executing it.

Real-Life Example



Everyday Language

The cat sat

↓ *break into words*

The

cat

sat

Sentence → Words



Programming Language

```
a = 1;
```

↓ *break into tokens*

a

=

1

;

Program → Tokens



Conclusion

Lexical analysis is the first phase of compilation — it converts raw source code into a clean stream of tokens that every later phase depends on.



Thank You